

\$unwind (aggregation stage)

Definition

`$unwind` Deconstructs an array field from the input documents to output a document for *each* element. Each output document is the input document with the value of the array field replaced by the element.

Compatibility

`$unwind` You can use `$unwind` for deployments hosted in the following environments:

- MongoDB Atlas: The fully managed service for MongoDB deployments in the cloud
- MongoDB Enterprise: The subscription-based, self-managed version of MongoDB
- MongoDB Community: The source-available, free-to-use, and self-managed version of MongoDB

Syntax

You can pass a field path operand or a document operand to unwind an array field.

Field Path Operand

You can pass the array field path to `$unwind`. When using this syntax, `$unwind` does not output a document if the field value is null, missing, or an empty array.

```
{ $unwind: <field path> }
```

When you specify the field path, prefix the field name with a dollar sign `$` and enclose in quotes.

Document Operand with Options

You can pass a document to `$unwind` to specify various behavior options.

```
{
  $unwind:
  {
    path: <field path>,
    includeArrayIndex: <string>,
    preserveNullAndEmptyArrays: <boolean>
  }
}
```

Field	Type	Description
path	string	Field path to an array field. To specify a field path, prefix the field name with a dollar sign `\$` and enclose in quotes.
includeArrayIndex	string	Optional. The name of a new field to hold the array index of the element. The name cannot start with a dollar sign `\$`.
preserveNullAndEmptyArrays	boolean	Optional. - If `true`, if `path` is missing or is an empty array, `\$unwind` omits the output field from the output document. If the value is `null`, the field remains `null`. - If `false`, if `path` is null, missing, or an empty array, `\$unwind` does not output a document. The default value is `false`.

Behaviors

Non-Array Field Path

When the value at `path` does not resolve to an array, `$unwind` behaves as follows:

- If the value is not missing, not `null`, and not an empty array, `$unwind` outputs a single document using the value as-is.
- If `includeArrayIndex` is specified, the index is `0` for array inputs and `null` for non-array inputs. Documents later in the list have an index greater than `0`.
- If the value is missing, `null`, or an empty array, `$unwind` follows the `preserveNullAndEmptyArrays` option. When `includeArrayIndex` is specified and the

document is preserved, the index is `null`.

Missing Field

If you specify a path for a field that does not exist in an input document or the field is an empty array, `$unwind`, by default, ignores the input document and will not output documents for that input document.

To output documents where the array field is missing, null or an empty array, use the `preserveNullAndEmptyArrays` option.

Examples

Unwind Array

In `mongosh`, create a sample collection named `inventory` with the following document:

```
db.inventory.insertOne({ _id: 1, item: "ABC1", sizes: [ "S", "M", "L" ] })
```

The following aggregation uses the `$unwind` stage to output a document for each element in the `sizes` array:

```
db.inventory.aggregate( [ { $unwind : "$sizes" } ] )
```

The operation returns the following results:

```
{ _id: 1, item: "ABC1", sizes: "S" }  
{ _id: 1, item: "ABC1", sizes: "M" }  
{ _id: 1, item: "ABC1", sizes: "L" }
```

Each document is identical to the input document except for the value of the `sizes` field which now holds a value from the original `sizes` array.

Missing or Non-array Values

Consider the `clothing` collection:

```
db.clothing.insertMany([
  { _id: 1, item: "Shirt", sizes: [ "S", "M", "L" ] },
  { _id: 2, item: "Shorts", sizes: [ ] },
  { _id: 3, item: "Hat", sizes: "M" },
  { _id: 4, item: "Gloves" },
  { _id: 5, item: "Scarf", sizes: null }
])
```

`$unwind` treats the `sizes` field as a single element array if:

- the field is present,
- the value is not null, and
- the value is not an empty array.

Expand the `sizes` arrays with `$unwind`:

```
db.clothing.aggregate( [ { $unwind: { path: "$sizes" } } ] )
```

The `$unwind` operation returns:

```
{ _id: 1, item: 'Shirt', sizes: 'S' },
{ _id: 1, item: 'Shirt', sizes: 'M' },
{ _id: 1, item: 'Shirt', sizes: 'L' },
{ _id: 3, item: 'Hat', sizes: 'M' }
```

- In document `"_id": 1`, `sizes` is a populated array. `$unwind` returns a document for each element in the `sizes` field.
- In document `"_id": 3`, `sizes` resolves to a single element array.
- Documents `"_id": 2`, `"_id": 4`, and `"_id": 5` do not return anything because the `sizes` field cannot be reduced to a single element array.

The `{ path: <FIELD> }` syntax is optional. The following `$unwind` operations are equivalent.

```
db.clothing.aggregate( [ { $unwind: "$sizes" } ] )
db.clothing.aggregate( [ { $unwind: { path: "$sizes" } } ] )
```

preserveNullAndEmptyArrays and includeArrayIndex

The `preserveNullAndEmptyArrays` and `includeArrayIndex` examples use the following collection:

```
db.inventory2.insertMany([
  { _id: 1, item: "ABC", price: Decimal128("80"), sizes: [ "S", "M", "L" ] },
  { _id: 2, item: "EFG", price: Decimal128("120"), sizes: [ ] },
  { _id: 3, item: "IJK", price: Decimal128("160"), sizes: "M" },
  { _id: 4, item: "LMN", price: Decimal128("10") },
  { _id: 5, item: "XYZ", price: Decimal128("5.75"), sizes: null }
])
```

preserveNullAndEmptyArrays

The following `$unwind` operation uses the `preserveNullAndEmptyArrays` option to include documents whose `sizes` field is null, missing, or an empty array.

```
db.inventory2.aggregate([
  { $unwind: { path: "$sizes", preserveNullAndEmptyArrays: true } }
])
```

The output includes those documents where the `sizes` field is null, missing, or an empty array:

```
{ _id: 1, item: "ABC", price: Decimal128("80"), sizes: "S" }
{ _id: 1, item: "ABC", price: Decimal128("80"), sizes: "M" }
{ _id: 1, item: "ABC", price: Decimal128("80"), sizes: "L" }
{ _id: 2, item: "EFG", price: Decimal128("120") }
{ _id: 3, item: "IJK", price: Decimal128("160"), sizes: "M" }
{ _id: 4, item: "LMN", price: Decimal128("10") }
{ _id: 5, item: "XYZ", price: Decimal128("5.75"), sizes: null }
```

includeArrayIndex

The following `$unwind` operation uses the `includeArrayIndex` option to include the array index in the output.

```
db.inventory2.aggregate( [
  {
    $unwind:
    {
      path: "$sizes",
      includeArrayIndex: "arrayIndex"
    }
  }
])
```

The operation unwinds the `sizes` array and includes the array index in the new `arrayIndex` field. If the `sizes` field does not resolve to a populated array but is not missing, null, or an empty array, the `arrayIndex` field is `null`.

```
{ _id: 1, item: "ABC", price: Decimal128("80"), sizes: "S", arrayIndex: Long(0)
{ _id: 1, item: "ABC", price: Decimal128("80"), sizes: "M", arrayIndex: Long(1)
{ _id: 1, item: "ABC", price: Decimal128("80"), sizes: "L", arrayIndex: Long(2)
{ _id: 3, item: "IJK", price: Decimal128("160"), sizes: "M", arrayIndex: null }
```

Group by Unwound Values

In `mongosh`, create a sample collection named `inventory2` with the following documents:

```
db.inventory2.insertMany([
  { _id: 1, item: "ABC", price: Decimal128("80"), sizes: [ "S", "M", "L" ] },
  { _id: 2, item: "EFG", price: Decimal128("120"), sizes: [ ] },
  { _id: 3, item: "IJK", price: Decimal128("160"), sizes: "M" },
  { _id: 4, item: "LMN" , price: Decimal128("10") },
  { _id: 5, item: "XYZ", price: Decimal128("5.75"), sizes: null }
])
```

The following pipeline unwinds the `sizes` array and groups the resulting documents by the unwound size values:

```

db.inventory2.aggregate( [
  // First Stage
  {
    $unwind: { path: "$sizes", preserveNullAndEmptyArrays: true }
  },
  // Second Stage
  {
    $group:
    {
      _id: "$sizes",
      averagePrice: { $avg: "$price" }
    }
  },
  // Third Stage
  {
    $sort: { "averagePrice": -1 }
  }
] )

```

The `$unwind` stage outputs a new document for each element in the `sizes` array. The stage uses the `preserveNullAndEmptyArrays` option to include in the output those documents where `sizes` field is missing, null or an empty array. This stage passes the following documents to the next stage:

```

{ _id: 1, item: "ABC", price: Decimal128("80"), sizes: "S" }
{ _id: 1, item: "ABC", price: Decimal128("80"), sizes: "M" }
{ _id: 1, item: "ABC", price: Decimal128("80"), sizes: "L" }
{ _id: 2, item: "EFG", price: Decimal128("120") }
{ _id: 3, item: "IJK", price: Decimal128("160"), sizes: "M" }
{ _id: 4, item: "LMN", price: Decimal128("10") }
{ _id: 5, item: "XYZ", price: Decimal128("5.75"), sizes: null }

```

The `$group` stage groups the documents by `sizes` and calculates the average price of each size. This stage passes the following documents to the next stage:

```

{ _id: "S", averagePrice: Decimal128("80") }
{ _id: "L", averagePrice: Decimal128("80") }
{ _id: "M", averagePrice: Decimal128("120") }
{ _id: null, averagePrice: Decimal128("45.25") }

```

The `$sort` stage sorts the documents by `averagePrice` in descending order. The operation returns the following result:

```
{ _id : "M", averagePrice: Decimal128("120") }  
{ _id : "L", averagePrice: Decimal128("80") }  
{ _id : "S", averagePrice: Decimal128("80") }  
{ _id : null, averagePrice: Decimal128("45.25") }
```

- `$group`
- `$sort`

Unwind Embedded Arrays

In `mongosh`, create a sample collection named `sales` with the following documents:


```

db.sales.insertMany( [
  {
    _id: "1",
    items: [
      {
        name: "pens",
        tags: [ "writing", "office", "school", "stationary" ],
        price: Decimal128("12.00"),
        quantity: Int32("5")
      },
      {
        name: "envelopes",
        tags: [ "stationary", "office" ],
        price: Decimal128("19.95"),
        quantity: Int32("8")
      }
    ]
  },
  {
    _id: "2",
    items: [
      {
        name: "laptop",
        tags: [ "office", "electronics" ],
        price: Decimal128("800.00"),
        quantity: Int32("1")
      },
      {
        name: "notepad",
        tags: [ "stationary", "school" ],
        price: Decimal128("14.95"),
        quantity: Int32("3")
      }
    ]
  }
])

```

The following operation groups the items sold by their tags and calculates the total sales amount per each tag.

```

db.sales.aggregate([
  // First Stage
  { $unwind: "$items" },

  // Second Stage
  { $unwind: "$items.tags" },

  // Third Stage
  {
    $group:
    {
      _id: "$items.tags",
      totalSalesAmount:
      {
        $sum: { $multiply: [ "$items.price", "$items.quantity" ] }
      }
    }
  }
])

```

The first `$unwind` stage outputs a new document for each element in the `items` array:

```

{ _id: "1", items: { name: "pens", tags: [ "writing", "office", "school", "stationary" ], price: 10, quantity: 100 }, totalSalesAmount: 1000 }
{ _id: "1", items: { name: "envelopes", tags: [ "stationary", "office" ], price: 5, quantity: 100 }, totalSalesAmount: 500 }
{ _id: "2", items: { name: "laptop", tags: [ "office", "electronics" ], price: 1000, quantity: 10 }, totalSalesAmount: 10000 }
{ _id: "2", items: { name: "notepad", tags: [ "stationary", "school" ], price: 10, quantity: 100 }, totalSalesAmount: 1000 }

```

The second `$unwind` stage outputs a new document for each element in the `items.tags` arrays:

```
{ _id: "1", items: { name: "pens", tags: "writing", price: Decimal128("12.00"),
{ _id: "1", items: { name: "pens", tags: "office", price: Decimal128("12.00"),
{ _id: "1", items: { name: "pens", tags: "school", price: Decimal128("12.00"),
{ _id: "1", items: { name: "pens", tags: "stationary", price: Decimal128("12.00"),
{ _id: "1", items: { name: "envelopes", tags: "stationary", price: Decimal128("12.00"),
{ _id: "1", items: { name: "envelopes", tags: "office", "price" : Decimal128("12.00"),
{ _id: "2", items: { name: "laptop", tags: "office", price: Decimal128("800.00"),
{ _id: "2", items: { name: "laptop", tags: "electronics", price: Decimal128("800.00"),
{ _id: "2", items: { name: "notepad", tags: "stationary", price: Decimal128("14.95"),
{ _id: "2", items: { name: "notepad", tags: "school", price: Decimal128("14.95") }
```

The `$group` stage groups the documents by the tag and calculates the total sales amount of items with each tag:

```
{ _id: "writing", totalSalesAmount: Decimal128("60.00") }
{ _id: "stationary", totalSalesAmount: Decimal128("264.45") }
{ _id: "electronics", totalSalesAmount: Decimal128("800.00") }
{ _id: "school", totalSalesAmount: Decimal128("104.85") }
{ _id: "office", totalSalesAmount: Decimal128("1019.60") }
```

The C# examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see [Get Started in the MongoDB .NET/C# Driver documentation](#).

The following `Movie` class models the documents in the `sample_mflix.movies` collection:

```

public class Movie
{
    public ObjectId Id { get; set; }

    public int Runtime { get; set; }

    public string Title { get; set; }

    public string Rated { get; set; }

    public List<string> Genres { get; set; }

    public string Plot { get; set; }

    public ImdbData Imdb { get; set; }

    public int Year { get; set; }

    public int Index { get; set; }

    public string[] Comments { get; set; }

    [BsonElement("lastupdated")]
    public DateTime LastUpdated { get; set; }
}

```

The C# classes on this page use Pascal case for their property names, but the field names in the MongoDB collection use camel case. To account for this difference, you can use the following code to register a `ConventionPack` when your application starts:

```

var camelCaseConvention = new ConventionPack { new CamelCaseElementNameConvention() };
ConventionRegistry.Register("CamelCase", camelCaseConvention, type => true);

```

`$unwind`

`Unwind()`

iterates over the `Genres` field in each input `Movie` document. For each value in the `Genres` field, the stage creates a new `Movie` document and populates its `Genres` field with the `Genres` value from the input document.

To use the MongoDB .NET/C# driver to add a `$unwind` stage to an aggregation pipeline, call the `Unwind()` method on a `PipelineDefinition` object.

The following example creates a pipeline stage that iterates over the `Genres` field in each input `Movie` document. For each value in the `Genres` field, the stage creates a new `Movie` document and populates its `Genres` field with the `Genres` value from the input document.

```
var pipeline = new EmptyPipelineDefinition<Movie>()
    .Unwind(m => m.Genres);
```

You can use an `AggregateUnwindOptions` object to customize the behavior of the `Unwind()` method. The following example performs the same operation as the previous example, but also includes the following options:

- `PreserveNullAndEmptyArrays` ensures that documents that contain an empty array in the `Genres` field are included in the output.
- The `IncludeArrayIndex` option adds a new field named `Index` to each output document. The value of this field is the array index of the `Genres` field's value in the input document's `Genres` array.

```
var pipeline = new EmptyPipelineDefinition<Movie>()
    .Unwind(m => m.Genres,
        new AggregateUnwindOptions<Movie>()
        {
            PreserveNullAndEmptyArrays = true,
            IncludeArrayIndex = new ExpressionFieldDefinition<Movie, int>(
                m => m.Index)
        });
```

The Node.js examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see [Get Started in the MongoDB Node.js driver documentation](#).

`$unwind`

iterates over the `genres` field in each input `movie` document. For each value in the `genres` field, the stage creates a new `movie` document and populates its `genres` field with the `genres` value from the input document

To use the MongoDB Node.js driver to add a `$unwind` stage to an aggregation pipeline, use the `$unwind` operator in a pipeline object.

The following example creates a pipeline stage that iterates over the `genres` field in each input `movie` document. For each value in the `genres` field, the stage creates a new `movie` document and populates its `genres` field with the `genres` value from the input document. The example then runs the aggregation pipeline:

```
const pipeline = [{ $unwind: "$genres" }];

const cursor = collection.aggregate(pipeline);
return cursor;
```

You can customize the behavior of the `$unwind` method. The following example performs the same operation as the previous example, but also includes the following options:

- `preserveNullAndEmptyArrays` ensures that documents that contain an empty array in the `genres` field are included in the output.
- `includeArrayIndex` adds a new field named `index` to each output document. The field contains the array index of the `genres` value in the input document's `genres` field.

```
const pipeline = [
  {
    $unwind: {
      path: "$genres",
      preserveNullAndEmptyArrays: true,
      includeArrayIndex: "index"
    }
  }
];

const cursor = collection.aggregate(pipeline);
return cursor;
```

Learn More

To learn more about related methods, see the `$group`, `$sum`, and `$multiply` guides.

To see how to use `$unwind` in a full example, see the [Unwind Arrays and Group Data tutorial](#).